User Manual — Photon calculator

---

1. Overview

- Purpose: estimate the flux density and photon counts output for a stellar observation given a star, telescope, and camera.
- Pipeline stages: top of atmosphere → ground level → after telescope → detected by sensor → ADU

---

2. Dependencies & Setup

- Required packages: numpy, pandas, matplotlib, scipy, astropy, requests
- Estructure of the directory:
    - objects.py – Main classes and functions.
    - test.py – Sample code.
    - test_flux_star.py – Sample code to validate the black body approximation calculations.
    - curves.py – Script with curves save in arrays. Here are save the atmospheric transmission, the aluminum reflectivity and the PCO quantum efficiency curves.
- Run from the repository root so relative paths resolve correctly

---

3. Core Classes (objects.py)

3.1 star(name, mag, temp, filter_type)

- name (default = 'Star'): identifier string

- magnitude (default = 9): apparent magnitude (Vega system)
- temp (default = ‘A0’): effective temperature in K or spectral type string (e.g. 'A2', 'G0', 'K0') — if spectral type string, the temperature is automatically extracted from a spectral type table.
- filter_type (default = ‘V’): filter to convert the magnitude to a flux density in [W /m2 /nm], two filters are supported V-band (‘V’) and the G filter for the GAIA 3 catalogue (‘G’).
- wl (default = ATM_WAV save in curves.py): Wavelengths for the atmospheric transmission in [nm].
- ext (default = ATM_TRANS save in curves.py): Atmospheric transmission in [%].

3.2 telescope(name, diameter, obscuration)

- name (default = ‘Telescope’): identifier string
- diameter (default = 2.54): primary mirror diameter in meters.
- obscuration (default = 0): fractional central obstruction (0–1)
- A : Effective collecting area is computed automatically
- wl (default = AL_BARE_WAV save in curves.py): Wavelengths for the reflectivity of the telescope coating in [nm].
- refl (default = AL_BARE_REFL save in curves.py): Reflectivity of the telescope coating in [%].

3.3 camera(name, full_well, bit_depth)

- name (default = ‘Camera’): identifier string
- full_well (default = 30000): fullwell capacity of the sensor in [e-]
- bit_depth (default = 16): Bit depth
- gain : Sensor gain in [e-/ADU]
- wl (default = PCO_QE_WAV save in curves.py): Wavelengths for the quantum efficiency of the sensor in [nm].

- ef (default = PCO_QE_EFF save in curves.py): Quantum efficiency of the sensor in [%].

---

4. Functions (objects.py)

Function	Purpose
compute_observation_flux(star, telescope, camera, exp_time, verbose)	Main entry point — returns photon counts at each stage and ADU count
ph_count_integral(...)	Integrates photon flux over the combined efficiency curve
flux_curves(star, wl, qe, atm, refl)	Returns spectral photon flux curves at each pipeline stage
combine_efficiency_curves(wl1,eff1, wl2,eff2, wl3,eff3)	Merges QE, atmosphere, and mirror reflectivity onto a common wavelength grid by interpolation
magnitude_2_flux(mag, type,F0)	Converts apparent magnitude to flux density ($W/m^2/nm$); supports 'V' for V-band and 'G' (Gaia DR3). F0 represents the reference value for Vega only used in 'V' type.
planck_law(lam, T)	Blackbody spectral radiance ($W/m^2/m$)

Function	Purpose
teff_from_spectral_type(sp_type)	Gets the Teff for the spectral type.

---

5. Running a Simulation (test3.py)

Step 1 — Define your objects

- Define star, telescope and camera objects with values required.

Step 2 — Call compute_observation_flux()

- Input the objects in compute_observation_flux() with an exposure time. A verbose option is included to get the Flux Density curves plot (Out of the atmosphere, at ground level, at the telescope output and detected by the sensor) and effiency curves plot (Telescope, Atmospheric Extinction and Camera Quantum Effiency).

Step 3 — Interpret the outputs

The function compute_observation_flux() returns 5 output variables that are shown in order on the next table:

Return value	Meaning
ph_out	Photons/s/m^2 above the atmosphere
ph_gnd	Photons/s/m^2 after atmospheric extinction
ph_tel	Photons/s after mirror reflectivity losses
ph_det	Total photons collected by the detector (e^-)

Return value	Meaning
adu	Detector signal in ADU

Step 4 — Run the script

Run the script. If compute_observation_flux() is nested in a for loop it is recommended to put verbose in 'False' to get the results faster. A script called '*test.py*' shows how to properly run the simulation.

---

6. Supported Spectral Types

- Accepts single-letter + digit codes: 'O5', 'B0', 'A0', 'F5', 'G0', 'K0', 'M0', etc.
- Unknown subtypes are linearly interpolated between the nearest entries in the built-in table
- A numeric Teff in Kelvin can always be passed directly instead
- The spectral types supported are in the next table:

Type	Teff (K)	Type	Teff (K)	Type	Teff (K)
O3	44900	A2	8800	K2	4900
O5	40900	A3	8600	K4	4600
O7	36500	A5	8100	K5	4440
O9	33000	A7	7600	K7	4050
B0	30000	A9	7370	M0	3870
B1	25400	F0	7200	M1	3700
B2	22000	F2	6890	M2	3550
B3	18700	F5	6440	M3	3410
B5	15400	F8	6200	M4	3200
B7	13000	G0	5920	M5	3030
B8	11400	G2	5770	M6	2850
B9	10500	G5	5660	M8	2500

A0	9700	G8	5440		
A1	9300	K0	5240		

7. Validation

The system is validated in 2 steps, the first testing the Blackbody approximation with flux density values measured from bright stars and second with raw images of the Latte campaign took in May 2024.

Step 1 — Blackbody approximation

Values of flux density in [Jy] are obtained using the online tool Vizier photometry viewer, this tool gets values from many catalogues at once, these values are transformed to [W/m^2 /nm] and compared to the values calculated with the blackbody approximation. Figure 1 show how the data is obtained from Vizier, as it can be seen a lot of data points are placed on the grid, ones that can defer from each other's, this is because there a lot of sources that are been used to get these data points, with instruments that are calibrated differently.

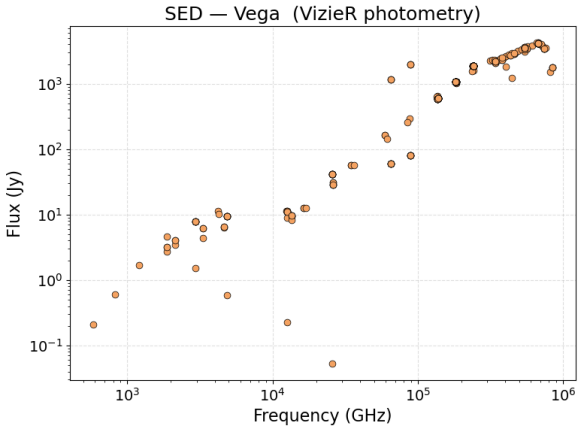

Figure 1. Flux density of Vega got from Vizier.

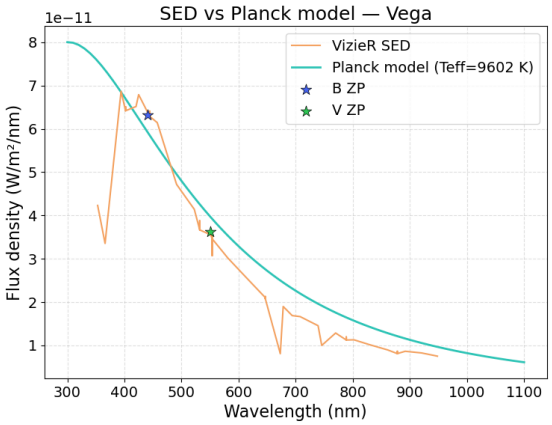

Figure 2. Flux density of Vega by black body approximation and data from catalogues. Stars are the values of flux get from the V-band and B-band magnitudes.

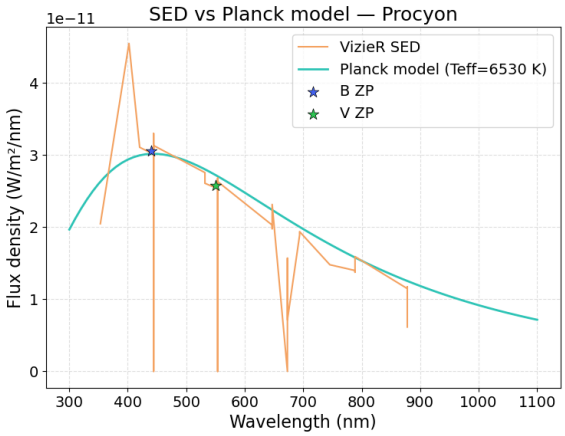

Figure 3. Flux density of Procyon by black body approximation and data from catalogues.

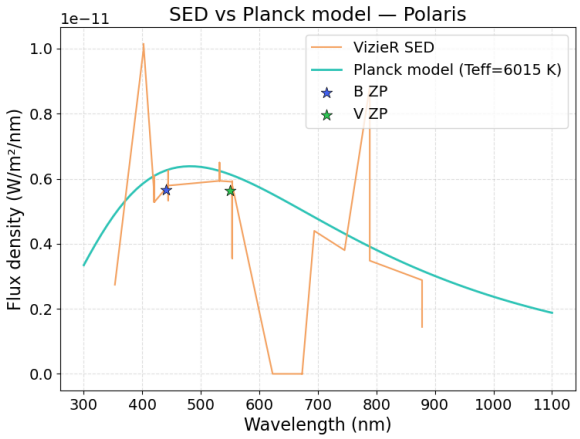

Figure 4. Flux density of Polaris by black body approximation and data from catalogues.

Figures 2, 3 and 4 show the comparison with the blackbody approximation. The measurements follow the theoretical curve for the most part where points exist. Also points with stars values are calculated directly from the magnitudes in V-band and B-band of each star, and are placed very near the theoretical curve. The script 'test_flux_star.py' shows how to get these plots, and it can be changed for any star by defining the star class and adding the B-band magnitude, although to use a bright star is advice cause the number of points obtained through the query decreases for dimmer stars.

Step 2 — Raw images 2024 campaign

In this section, the theoretical blackbody approximation is compared with photometry calculus of the stars captured in May 2024. The astrometry.net was used to get the positions and identify stars in the scene as it can be seen on the Figure 5.

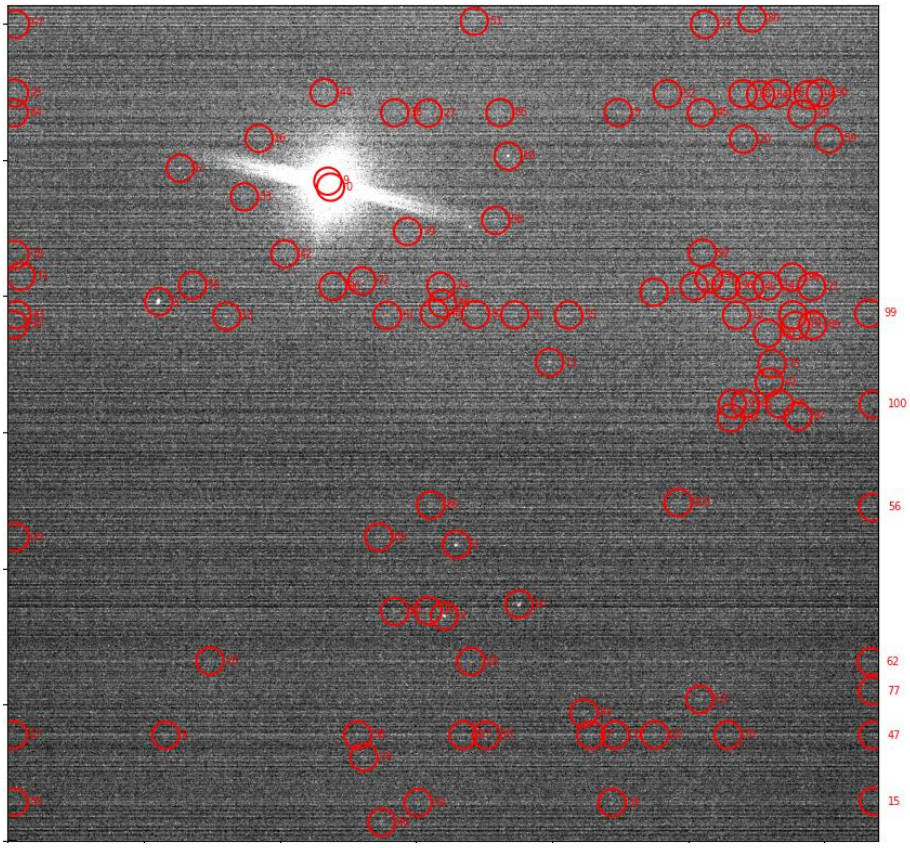

Figure 5. Example of a raw image with the stars found using astrometry.net

With the astrometry done is possible to find the magnitudes and temperatures to calculate the black body approximation for each star and the positions in the images to do the photometry calculations. The latter is done calculating the size of the star and subtracting the background. The Figure 6 shows the results of comparing each theoretical value with the calculated from the raw image in different images. As it can be seen, points spread a lot increasing the difference between the theoretical and calculated values, a percentage of this discrepancy can be explained by vignetting happening in the telescope used. To check if The ratio between the calculated values and the theoretical values are show in the Figure 7, this demonstrate that a bigger difference happens while the stars are farther from the center of the sensor.

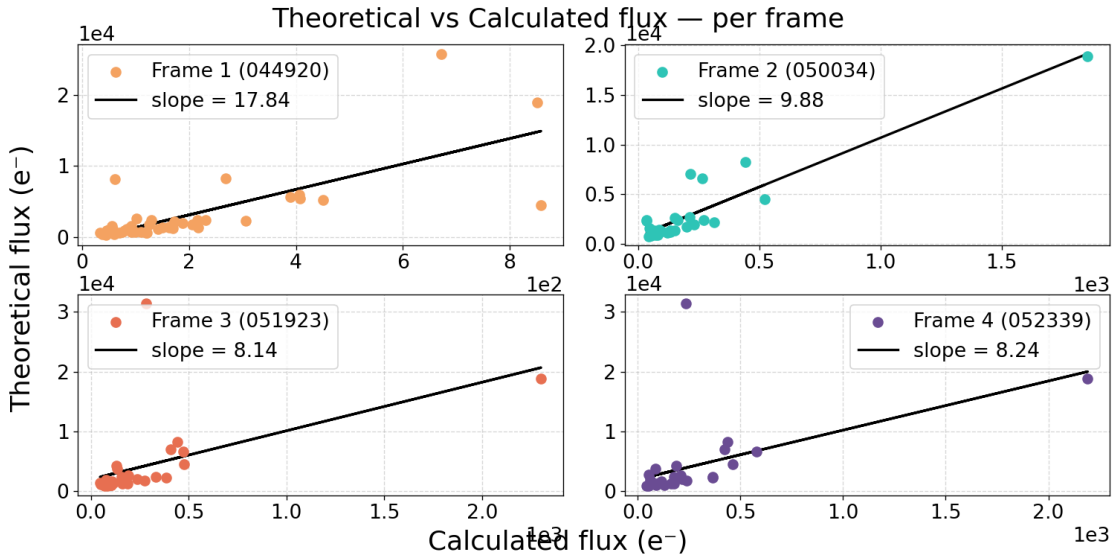

Figure 6. Flux densities from the black body approximation (Theoretical) vs calculated using photometry (Calculated) of 4 different raw images.

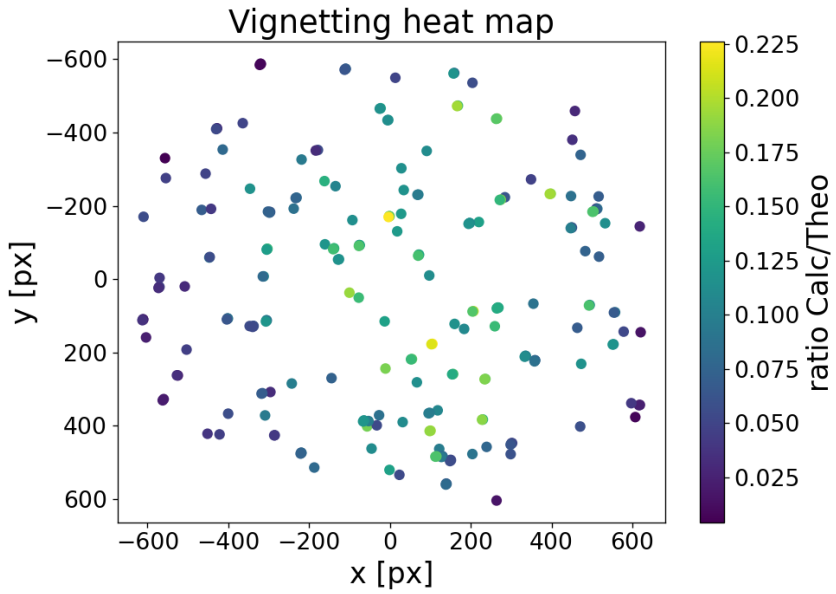

Figure 7. Ratio Calculated/Theoretical heat map.

The Figure 8 and 9 shows how the difference between theoretical values and calculated values decreases by only using the stars that are 200 pixels from the center for 8 raw images.

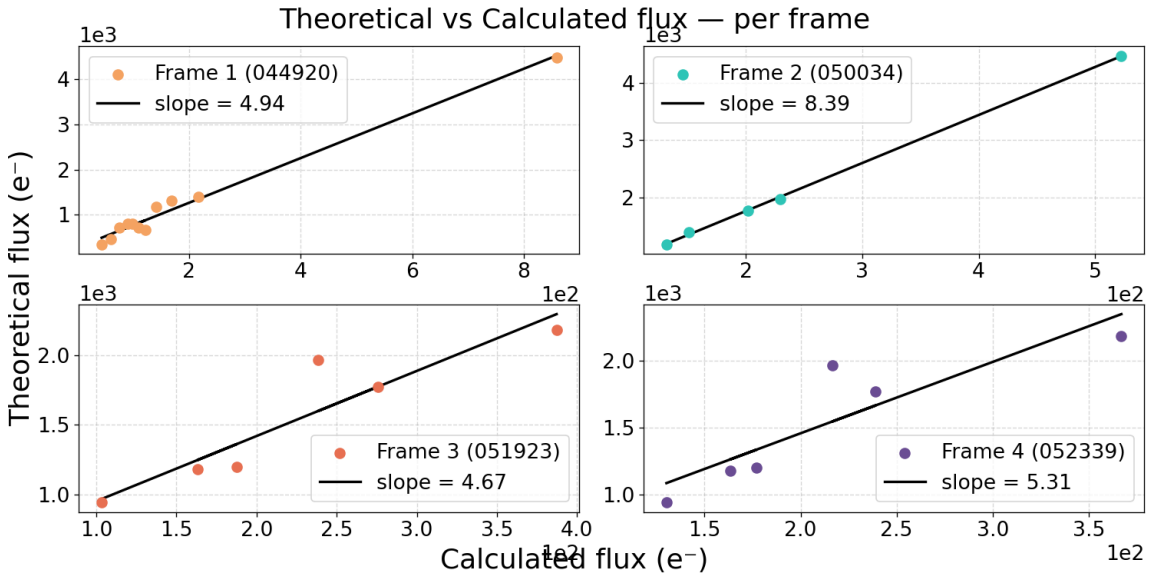

Figure 8. Flux densities from the black body approximation (Theoretical) vs calculated using photometry (Calculated) of 4 different raw images.

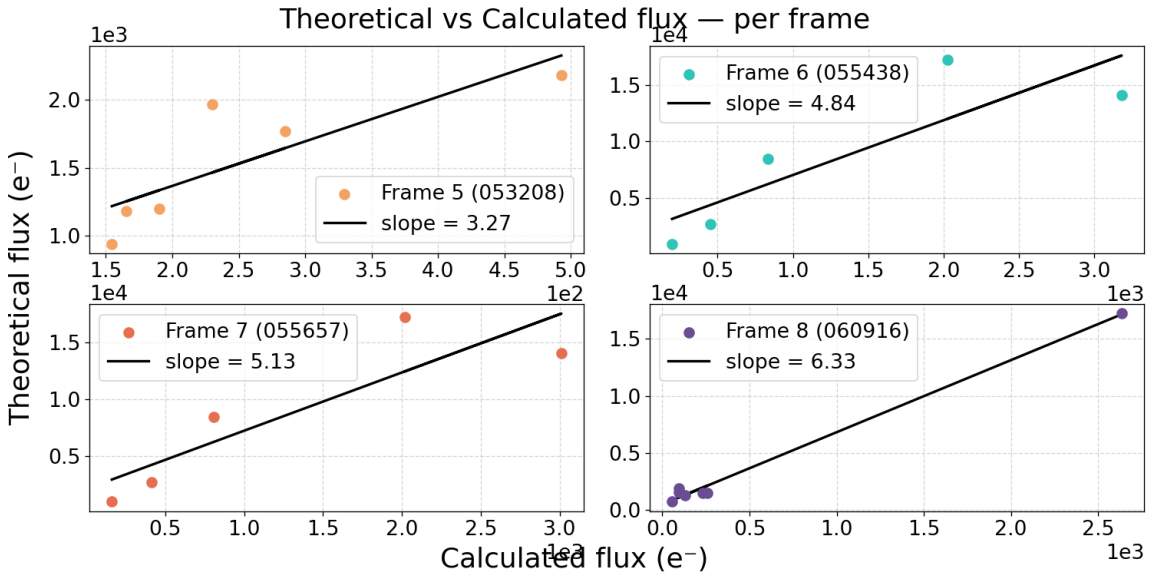

Figure 9. Flux densities from the black body approximation (Theoretical) vs calculated using photometry (Calculated) of 4 different raw images.

8. Notes & Limitations

- Atmospheric extinction uses a fixed tabulated La Palma profile; no zenith-angle scaling yet
- Mirror reflectivity model assumes bare enhanced-aluminum coating applied twice for a 2-mirror telescope.
- Camera QE is always loaded from the PCO CSV file regardless of which camera object is created
- The curves before mentioned can be changed by inputting new ones in its respective classes (star for the atmospheric transmission, telescope for the material reflectivity and camera for the quantum efficiency).